

While Loops

Looping Control Flow

Looping control flow executes a chunk of code more than once.

It jumps back so that you can do something again

Since you don't usually want infinite loops, it typically has some conditional logic to know when to stop looping

A New AST Node

We need to introduce a new `While` statement node to handle this control flow

It stores the *condition* and the body of the loop. Separating expressions and statements makes it clear that the condition is an expression and the body is a statement

```
1  tool/GenerateAst.java
2  in main()
```

```
1      "Print      : Expr expression",
2      "Var        : Token name, Expr initializer",
3      "While      : Expr condition, Stmt body"
```

Updating the Grammar

We need to add the `while` statement to our grammar

```
1  statement  → exprStmt
2              | ifStmt
3              | printStmt
4              | whileStmt
5              | block ;
6
7  whileStmt  → "while" "(" expression ")" statement ;
```

Updating the Parser

Over in the parser, we follow the same process we used for `if` statements by updating our `statement()` rule to detect and match the leading keyword

```
1  lox/Parser.java
2  in statement()
```

```
1      if (match(IF)) return ifStatement();
2      if (match(PRINT)) return printStatement();
3      if (match(WHILE)) return whileStatement();
4      if (match(LEFT_BRACE)) return new Stmt.Block(block());
```

Parsing While Loops

The `whileStatement()` method delegates the real work of parsing the condition and the body

It hews closely to the grammar, consuming the mandatory parentheses around the condition expression

```
1  lox/Parser.java
2  after ifStatement()
```

```
1  private Stmt whileStatement() {
2      consume(LEFT_PAREN, "Expect '(' after 'while'.");
3      Expr condition = expression();
4      consume(RIGHT_PAREN, "Expect ')' after condition.");
5      Stmt body = statement();
6
7      return new Stmt.While(condition, body);
8  }
```

Interpreting While Loops

Now for the interpreter. We evaluate the condition expression first

If it's truthy, we execute the body. We keep looping as long as the condition remains truthy!

```
1  lox/Interpreter.java
2  in class Interpreter
```

```
1  @Override
2  public Void visitWhileStmt(Stmt.While stmt) {
3      while (isTruthy(evaluate(stmt.condition))) {
4          execute(stmt.body);
5      }
6      return null;
7  }
```

Turing Completeness

Like `if` statements, the interpreter implementation is a thin wrapper around Java's own code

We use Java's `while` loop to execute the Lox `while` loop

It's simple, but it finally makes Lox **Turing-complete**, letting us write programs where the running time isn't strictly bound by the length of the source code